ANEXO B. MANUAL DE INSTALACIÓN, DESPLIEGUE Y MANTENIMIENTO DE VIDIA

Referencia técnica y comprobaciones: 12 de octubre de 2026.

B.1. Objetivo

Establecer los requisitos y procedimientos para obtener, configurar, compilar, ejecutar y mantener Vidia. El manual distingue la configuración versionada, las herramientas disponibles, las operaciones comprobadas en un entorno local aislado y las consultas públicas del servicio desplegado.

Las compilaciones se realizaron sobre copias temporales del árbol de trabajo. Se utilizó PostgreSQL temporal en 127.0.0.1:55439, sin importar credenciales ni datos de producción. Las comprobaciones públicas fueron consultas sin autenticación; no se desplegó una versión ni se modificó el servicio publicado.

Los comandos de consola de este documento utilizan PowerShell. Ejecútelos desde las carpetas indicadas y adapte solo los valores locales que se señalan como ejemplos. Los resultados conservados se encuentran en docs/documentoFinal/evidencias_anexo_b_20261012/. Los marcadores son espacios de evidencia pendiente, no resultados ejecutados.

B.2. Componentes de la solución

Componente	Ruta o configuración	Responsabilidad
Backend	backend/pom.xml; src/main/java	Java 21 / Spring Boot 3.3.0; monolito modular con controladores, servicios, repositorios y entidades.

Componente	Ruta o configuración	Responsabilidad
Web	frontend/package.json; angular.json	Angular 21; clientes participante, organizador y administrador.
Android	flutter/VidiaApp/pubspec.yaml	Flutter/Dart; aplicación de participante, sin gestión administrativa ni organizadora móvil.
Persistencia	backend/src/main/resources/application.yml	PostgreSQL, JPA/Hibernate y migraciones Flyway.
Contenedor backend	backend/Dockerfile	Compilación Maven y ejecución del JAR en Java 21; no son microservicios.
Alojamiento web	render.yaml	Servicio estático Render; reescritura de rutas de Angular a index.html.
Archivos	app.storage; servicios de pagos/eventos	Proveedor local o Supabase Storage compatible con S3: comprobantes y QR de pago. El PDF de certificado se genera al descargarlo; no se presume almacenado en Supabase.
Correo	BrevoApiCorreoProveedor; CorreoServicio	Brevo API HTTP con JSON. SMTP existe como alternativa seleccionable, no como descripción de la integración API.
Distribución Android	GitHub Releases, etiqueta v1.0.0	Publicación del instalable existente; no se publicó un artefacto en esta revisión.

Los módulos backend localizados son usuarios, categorías, eventos, inscripciones, pagos, sesiones, asistencias, certificados y reportes. Los roles técnicos son

ADMINISTRADOR, ORGANIZADOR y USUARIO. La cuenta administrativa es exclusiva; el organizador conserva USUARIO.

B.3. Requisitos previos

Herramienta	**Referencia / instalada**	**Función y comprobación**
JDK	Proyecto 21; Temurin 21.0.11 instalado	Compilar y ejecutar backend. java -version; javac -version. Android configura bytecode JVM 17, distinto del nivel Java 21 del backend.
Maven	Dockerfile 3.9.9; local 3.10.0	mvn -version. No se localizó Maven Wrapper en backend; disponer de Maven en PATH.
Node.js / npm	24.16.0 / 11.6.3 locales	node --version; npm --version. packageManager declara npm@11.6.3.
Angular	^21.0.0 declarado; 21.2.17 resuelto	Después de npm ci: npx --no-install ng version. No requiere CLI global. Engines: ^20.19.0, ^22.12.0 o >=24.0.0 de Node.
Flutter / Dart	Flutter 3.44.8; Dart 3.12.2 instalados	flutter --version; dart --version. pubspec exige Dart >=3.12.0 <4.0.0; Flutter SDK no está fijado por pubspec.

Herramienta	Referencia / instalada	Función y comprobación
Android	SDK 36.0.0; minSdk 24	flutter doctor. Gradle 9.1.0, AGP 9.0.1 y Kotlin 2.3.20 declarados. Licencias pendientes observadas, aunque el APK debug compiló.
PostgreSQL	16 en Compose; binarios 16.15 locales	psql --version; pg_dump --version. La versión productiva no se obtuvo.
Docker / Compose	Cliente 26.1.1 / Compose 2.27.0	docker version; docker compose version. Daemon no disponible; opcional para desarrollo nativo, requerido para probar imágenes.
Git	Referencia f6891ef3ffb5a89a31e323d37a7ae9d2f734725c	git --version; git rev-parse HEAD. El árbol local contiene cambios previos no confirmados.

No se infieren versiones productivas de Java, PostgreSQL, Angular o Flutter a partir de estos datos locales. Spring Boot 3.3.0 se declara en pom.xml; el JAR comprobado contiene Hibernate 6.5.2.Final, Flyway 10.10.0, PostgreSQL JDBC 42.7.3 y Spring Security 6.3.0.

[INSERTAR CAPTURA: versiones instaladas de las herramientas.]

Capturar la consola real con java -version, mvn -version, node --version, npm --version, flutter --version y dart --version. Los resultados ya están en versiones_instaladas.json; no recrear una terminal.

B.4. Obtención del código fuente

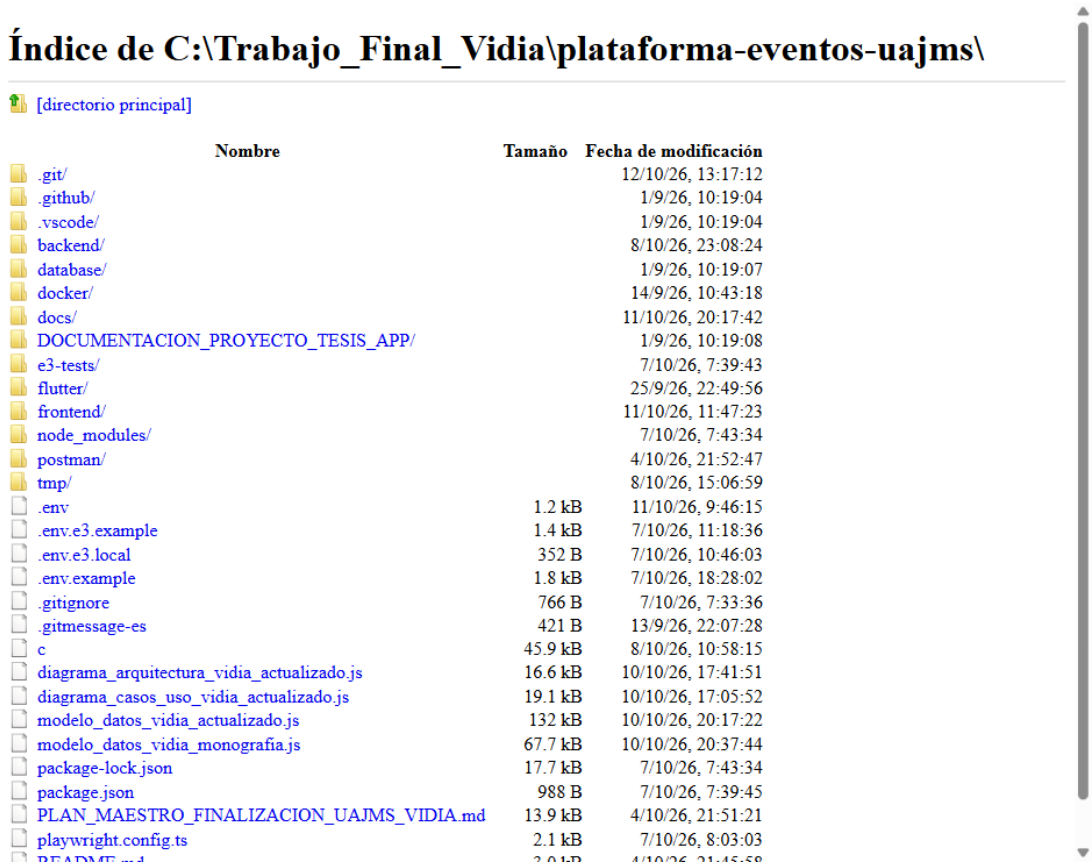
1. Clone el repositorio en una carpeta de trabajo nueva. No reemplace un directorio con cambios locales.

2. Acceda a la carpeta creada por Git y compruebe los manifiestos.

3. Registre commit, rama y estado antes de instalar o liberar una versión.

```
git clone https://github.com/felix-lamas/trabajo-final-diplomado.git
cd trabajo-final-diplomado
git rev-parse HEAD
git branch --show-current
git status --short
Test-Path backend/pom.xml
Test-Path frontend/package.json
Test-Path frontend/angular.json
Test-Path flutter/VidiaApp/pubspec.yaml
```

El checkout examinado ya existía en plataforma-eventos-uajms y estaba en la rama main. No se ejecutó una nueva clonación. El commit base fue f6891ef3ffb5a89a31e323d37a7ae9d2f734725c; no identifica por sí solo el código ensayado porque había cambios locales preexistentes. integridad_entrada.json conserva las huellas de los archivos. Para repetir exactamente la revisión, se necesita ese estado de fuentes o una revisión posterior que lo incorpore.



Índice de C:\Trabajo_Final_Vidia\plataforma-eventos-uajms

[directorio principal]

Nombre	Tamaño	Fecha de modificación
.git/		12/10/26, 13:17:12
.github/		1/9/26, 10:19:04
.vscode/		1/9/26, 10:19:04
backend/		8/10/26, 23:08:24
database/		1/9/26, 10:19:07
docker/		14/9/26, 10:43:18
docs/		11/10/26, 20:17:42
DOCUMENTACION_PROYECTO_TESIS_APP/		1/9/26, 10:19:08
e3-tests/		7/10/26, 7:39:43
flutter/		25/9/26, 22:49:56
frontend/		11/10/26, 11:47:23
node_modules/		7/10/26, 7:43:34
postman/		4/10/26, 21:52:47
tmp/		8/10/26, 15:06:59
.env	1.2 kB	11/10/26, 9:46:15
.env.e3.example	1.4 kB	7/10/26, 11:18:36
.env.e3.local	352 B	7/10/26, 10:46:03
.env.example	1.8 kB	7/10/26, 18:28:02
.gitignore	766 B	7/10/26, 7:33:36
.gitmessage-es	421 B	13/9/26, 22:07:28
c	45.9 kB	8/10/26, 10:58:15
diagrama_arquitectura_vidia_actualizado.js	16.6 kB	10/10/26, 17:41:51
diagrama_casos_uso_vidia_actualizado.js	19.1 kB	10/10/26, 17:05:52
modelo_datos_vidia_actualizado.js	132 kB	10/10/26, 20:17:22
modelo_datos_vidia_monografia.js	67.7 kB	10/10/26, 20:37:44
package-lock.json	17.7 kB	7/10/26, 7:43:34
package.json	988 B	7/10/26, 7:39:45
PLAN_MAESTRO_FINALIZACION_UAJMS_VIDIA.md	13.9 kB	4/10/26, 21:51:21
playwright.config.ts	2.1 kB	7/10/26, 8:03:03

Estructura observada del checkout local, no de una clonación nueva
Fuente: elaboración propia a partir de la consulta real. 12/10/2026 14:15 (Bolivia).

B.5. Preparación de la base de datos

B.5.1. Preparación nativa local

La URL predeterminada compone DB_HOST=localhost, DB_PORT=5432 y DB_NAME=uajms_eventos. DB_USER y DB_PASS no tienen valor predeterminado. Puede sustituirse la URL completa mediante DB_URL. No copie una conexión de Render al entorno de pruebas.

1. Instale o identifique PostgreSQL y sus herramientas psql, pg_dump y pg_restore. En este equipo están en C:\Program Files\PostgreSQL\16\bin y no estaban en PATH.

2. Conéctese con el administrador de la instancia local. Cree una base separada y una cuenta sin privilegios globales de administración.

3. Los siguientes nombres de usuario son un ejemplo local; la base uajms_eventos corresponde al nombre predeterminado del proyecto. Introduzca la contraseña mediante el prompt de psql.

```
$env:PATH += ";C:\Program Files\PostgreSQL\16\bin"
$adminPg = Read-Host "Administrador PostgreSQL local"
psql -h 127.0.0.1 -U $adminPg -d postgres
```

```
CREATE ROLE vidia_local LOGIN NOSUPERUSER
  NOCREATEDB NOCREATEROLE;
\password vidia_local
CREATE DATABASE uajms_eventos OWNER vidia_local;
```

Como propietario de su base, el usuario de Vidia puede aplicar DDL de sus migraciones. Si la institución separa migración y ejecución con roles diferentes, el procedimiento de privilegios debe diseñarse y verificarse expresamente; no fue el esquema ensayado.

B.5.2. Alternativa Docker Compose

```
docker compose --env-file .env `
  -f docker/docker-compose.yml config --services
docker compose --env-file .env `
  -f docker/docker-compose.yml up -d database pgadmin
```

Desde la raíz, prepare .env con DB_NAME, DB_USER, DB_PASS, PGADMIN_EMAIL y PGADMIN_PASSWORD. El descriptor usa postgres:16-alpine, puertos 5432:5432 y 5050:80, volumen db_data y el directorio database/scripts. Este directorio contiene contexts; no sustituye las migraciones del backend. El descriptor se validó estáticamente y devolvió database y pgadmin. No se ejecutó up porque el daemon no estaba disponible.

Antes de ejecutar Compose, compruebe colisiones de puertos y la política de exposición de la red local. No utilice down -v sobre volúmenes con información que deba conservarse.

[INSERTAR CAPTURA: contenedores PostgreSQL y pgAdmin en ejecución.]

Mostrar docker compose ps y disponibilidad de la base aislada, sin imprimir variables ni credenciales. La validación estática no acredita contenedores iniciados.

B.5.3. Migraciones y esquema

Flyway está habilitado y usa classpath:db/migration. Las migraciones V1–V5 se aplican al arranque; baseline-on-migrate=true y baseline-version=0 están configurados. No use el baseline automático para dar por válida una base histórica sin revisar su esquema y probar una copia.

V1 y V2 tienen lógica aditiva para tablas existentes; V3 crea/reconcilia el esquema canónico, V4 agrega invitaciones administrativas y V5 completa solicitudes de organizador. Los comentarios antiguos sobre creación de tablas por Hibernate no describen todo el comportamiento vigente de V3–V5.

La configuración predeterminada de JPA_DDL_AUTO es update. El ensayo se ejecutó con validate para comprobar que Flyway prepara el esquema antes de JPA. Se observaron cinco migraciones correctas, 19 tablas de dominio y una tabla técnica flyway_schema_history.

El clúster temporal de ensayo usó autenticación trust limitada a loopback para las pruebas opt-in. No se comprobó autenticación PostgreSQL mediante contraseña en ese clúster; trust no es una configuración propuesta para operación institucional.

```
SELECT version, description, success
FROM flyway_schema_history ORDER BY installed_rank;
SELECT count(*) FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE';
```

Resultado local: V1–V5 aplicadas y 20 tablas contando Flyway. Esta comprobación no describe el estado de PostgreSQL productivo.

[INSERTAR CAPTURA: historial de migraciones locales.]

Capturar la consulta real de flyway_schema_history. El resultado está conservado en migraciones_y_restauracion.json; no mostrar una conexión productiva.

B.6. Instalación y ejecución del backend

B.6.1. Configuración y compilación

Ejecute desde backend/, donde está pom.xml. application.yml importa opcionalmente ../.env como archivo de propiedades. También admite variables del proceso. No imprima valores sensibles ni incluya .env en Git. Los ejemplos siguientes solo preparan un entorno local; las variables completas se agrupan en B.13.

```
cd backend
$env:DB_HOST = "127.0.0.1"
$env:DB_PORT = "5432"
$env:DB_NAME = "uajms_eventos"
$env:DB_USER = "vidia_local"
$clave = Read-Host "Clave de la base local" -AsSecureString
$env:DB_PASS = [Net.NetworkCredential]::new("", $clave).Password
$bytes = New-Object byte[] 48
$rng = [Security.Cryptography.RandomNumberGenerator]::Create()
$rng.GetBytes($bytes)
$env:JWT_SECRET = [Convert]::ToBase64String($bytes)
$rng.Dispose()
$env:SPRING_PROFILES_ACTIVE = "dev"
$env:JPA_DDL_AUTO = "validate"
$env:APP_STORAGE_PROVIDER = "local"
mvn -B clean verify
```

No registre JWT_SECRET ni DB_PASS. El ejemplo no produce una clave de firma Android ni cambia secretos del despliegue. Para evitar importar un .env de otro ambiente durante una comprobación aislada, establezca SPRING_CONFIG_IMPORT vacío en ese proceso.

Resultado ejecutado: mvn -B clean verify terminó con código 0. Se ejecutaron 591 pruebas: 590 aprobadas, 1 omitida, 0 fallos y 0 errores; 66 suites. Se generó target/eventos-uajms-0.0.1-SNAPSHOT.jar (86,053,010 bytes). La imagen Docker usa -DskipTests; una construcción de imagen no sustituye esta suite.

Las pruebas de integración opt-in requieren VIDIA_TEST_PG_URL=jdbc:postgresql://127.0.0.1:55439/postgres y el usuario vidia_test en un clúster descartable. Ambas clases crean bases temporales propias. No habilite esa variable para una base compartida.

RUN_SUPABASE_STORAGE_SMOKE permaneció deshabilitada, por lo que no se escribieron objetos externos.

[INSERTAR CAPTURA: compilación y pruebas del backend.]

Capturar el BUILD SUCCESS y resumen del runner real. Reportes conservados: backend_verify.txt, backend_junit_resumen.json y junit_backend/. No se obtuvo una captura de terminal gráfica.

B.6.2. Arranque y detención

```
mvn -B spring-boot:run
# Alternativa después de compilar:
java -jar target/eventos-uajms-0.0.1-SNAPSHOT.jar
```

SERVER_PORT tiene prioridad sobre PORT; si ambos faltan, se utiliza 8080. Se comprobó mvn spring-boot:run con perfil dev y después java -jar con dev,demo. En ambos se utilizó la base temporal; no se probó el arranque contra producción.

Con perfil dev solamente no se ejecuta DatosInicialesSeed. Para disponer de datos y cuentas ficticias en una base de demostración autorizada, añada demo y proporcione DEMO_PASSWORD de forma privada. No active demo en una base institucional solo para probar el manual.

```
$env:SPRING_PROFILES_ACTIVE = "dev,demo"
$claveDemo = Read-Host "Clave de demostración local" -AsSecureString
$env:DEMO_PASSWORD = [Net.NetworkCredential]::new("", $claveDemo).Password
java -jar target/eventos-uajms-0.0.1-SNAPSHOT.jar
```

En la base temporal se comprobó el seed y el acceso local: administrador con solo ADMINISTRADOR y organizador con USUARIO + ORGANIZADOR. No se incluyen sus contraseñas ni tokens en las evidencias. DatosInicialesSeed no se modificó.

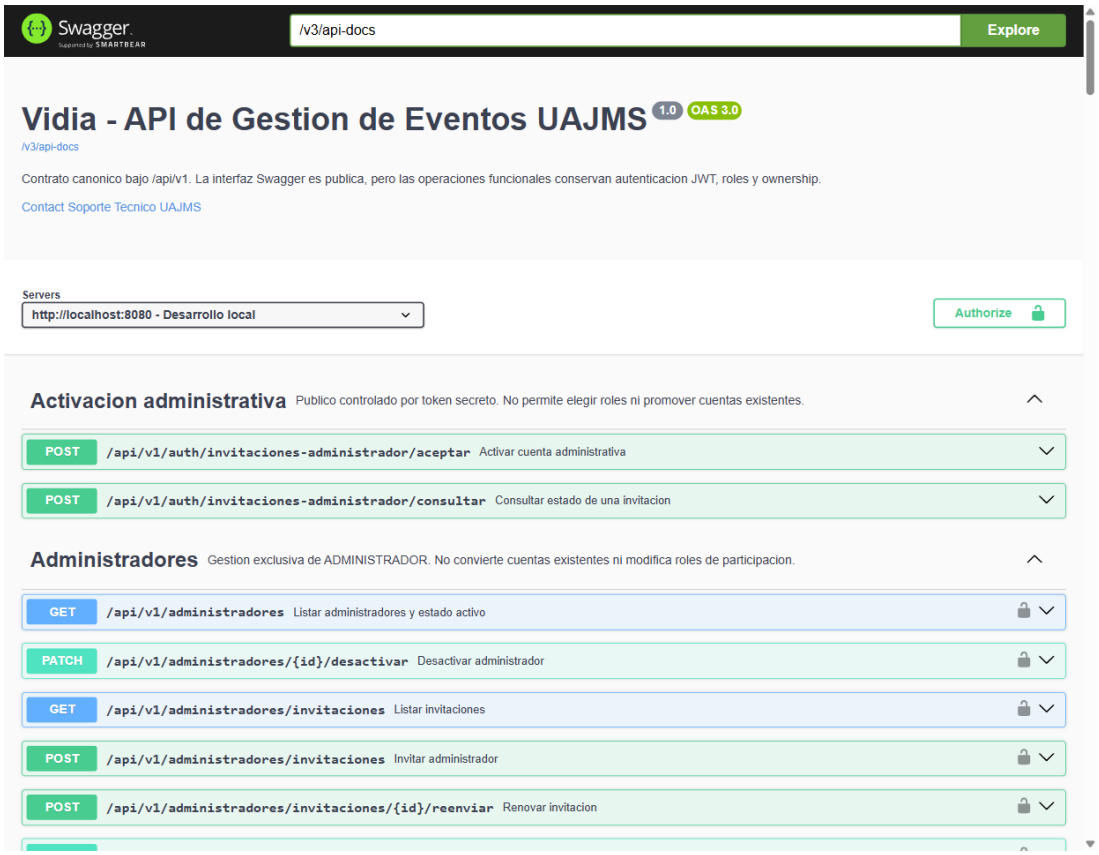
Detenga un proceso interactivo con Ctrl+C. Para servicios administrados, utilice la herramienta que los inició y confirme la liberación del puerto. Nunca detenga todos los procesos java o postgres para solucionar una colisión.

Incidencia conservada: un primer intento de java -jar falló porque 8080 aún estaba ocupado por nuestro arranque Maven. Se verificó la pertenencia del proceso

temporal, se detuvo ese proceso y el JAR arrancó correctamente. No se cambió código para resolverlo.

[INSERTAR CAPTURA: arranque efectivo del backend.]

Mostrar Tomcat started on port 8080 y la URL local de salud. Logs reales: backend_arranque.txt, backend_jar_seed.txt y el primer fallo backend_jar_puerto_ocupado.txt.



Swagger del backend iniciado en la base aislada
Fuente: elaboración propia a partir de la consulta real. 12/10/2026 14:15 (Bolivia).

B.7. Instalación y ejecución del frontend web

Ejecute desde frontend/. package-lock.json existe y npm@11.6.3 se declara en package.json. El procedimiento reproducible es npm ci, no una resolución nueva mediante npm install. No se requiere instalar Angular CLI globalmente.

```
cd frontend
npm ci
npx --no-install ng version
npm test -- --watch=false
npm run build
npm start
# Equivalente para desarrollo:
npx --no-install ng serve
```

En el ensayo se usó npm ci --no-audit --fund=false, sin modificar el archivo de bloqueo original. npm test -- --watch=false aprobó 316 pruebas en 57 archivos. npm run build terminó con código 0; se generó dist/frontend/browser. Se observaron advertencias de presupuestos de bundle/estilos; no impidieron la construcción.

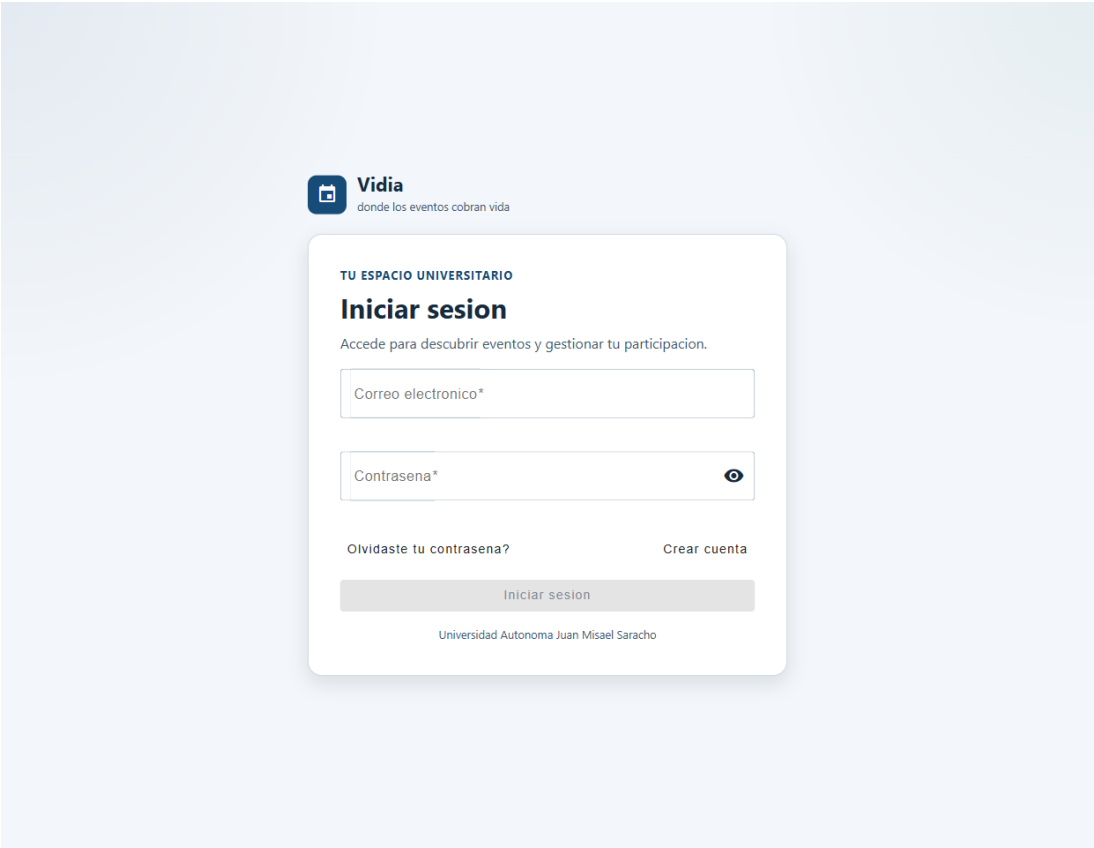
npm start ejecuta ng serve, con configuración development. Se comprobó npm start -- --host 127.0.0.1 --port 4200 y se cargaron acceso y catálogo. Detenga el servidor con Ctrl+C. El puerto 4200 se verificó en esta ejecución; no debe suponerse si el comando utiliza otro puerto.

environment.dev.ts usa http://localhost:8080/api/v1. environment.ts usa https://trabajo-final-diplomado.onrender.com/api/v1. angular.json sustituye el archivo en development; la compilación predeterminada es production. Una variable de Render no cambia automáticamente esa constante compilada.

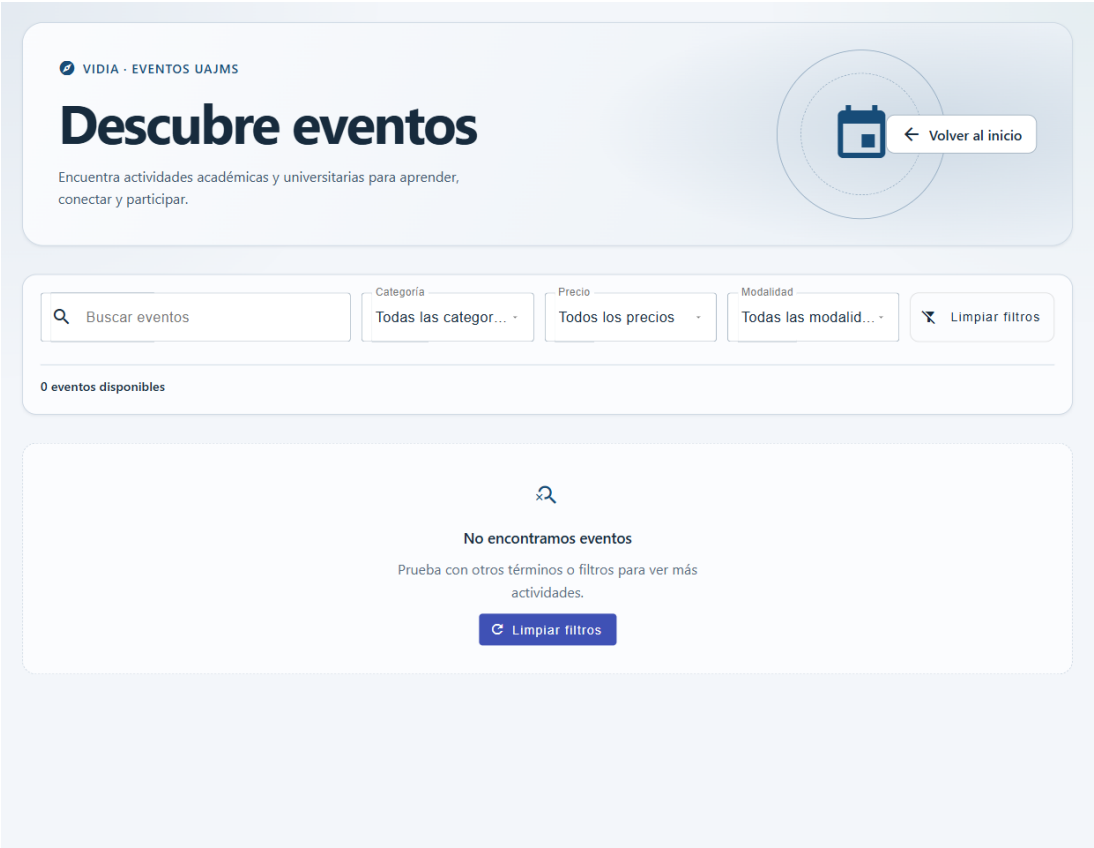
Si cambia el puerto del backend, ajuste la configuración de desarrollo en su copia autorizada y vuelva a compilar. localhost dentro de un contenedor o teléfono no identifica necesariamente al equipo que aloja el backend. Verifique solicitudes y CORS antes de atribuir el fallo a la interfaz.

[INSERTAR CAPTURA: instalación, pruebas y build de Angular.]

Capturar la consola real de npm ci, el resumen 316/316 y el resultado del build. Los registros están en angular_npm_ci.txt, angular_test.txt y angular_build.txt.



Interfaz web de desarrollo servida desde la copia temporal
Fuente: elaboración propia a partir de la consulta real. 12/10/2026 13:34 (Bolivia).



Catálogo local antes de habilitar datos de demostración
Fuente: elaboración propia a partir de la consulta real. 12/10/2026 13:34 (Bolivia).

B.8. Preparación de la aplicación móvil

B.8.1. Dependencias y pruebas

```
cd flutter/VidiaApp
flutter --version
dart --version
flutter doctor
flutter pub get
flutter test
flutter analyze
```

Se verificaron Flutter 3.44.8 y Dart 3.12.2. pubspec.lock se conservó en el proyecto original; pub get se ejecutó únicamente en la copia temporal. flutter doctor devolvió código 0, pero señaló licencias Android pendientes. No se aceptaron licencias automáticamente ni se cambiaron herramientas.

flutter test --reporter expanded terminó correctamente: 139 pruebas aprobadas. flutter analyze devolvió código 1 con 17 avisos, todos informativos. No se presenta el análisis como una ejecución sin observaciones. No se alteró código para eliminarlas.

Un responsable autorizado puede revisar las licencias con flutter doctor --android-licenses. Ese comando de aceptación no se ejecutó; el doctor conserva su advertencia.

[INSERTAR CAPTURA: Flutter doctor, pruebas y análisis.]

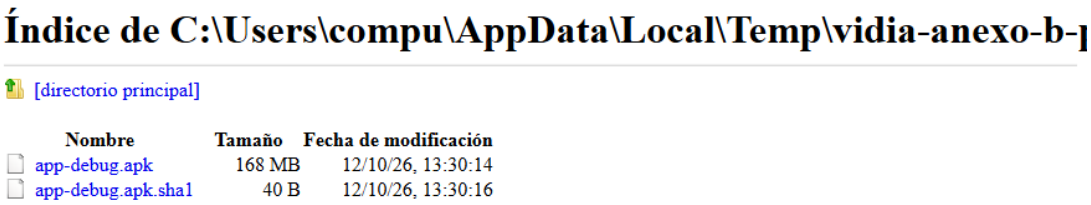
Capturar el resultado real de doctor, las 139 pruebas y los 17 avisos de análisis. Los archivos flutter_doctor.txt, flutter_test.txt y flutter_analyze.txt conservan la ejecución.

B.8.2. URL del backend y APK de desarrollo

ApiConfig utiliza String.fromEnvironment("API_BASE_URL"). La URL debe terminar exactamente en /api/v1. Sin parámetro, Android utiliza http://10.0.2.2:8080/api/v1 para el emulador; web y escritorio usan localhost. En un teléfono físico se necesita una dirección accesible desde el dispositivo o un entorno público autorizado, no 10.0.2.2.

```
flutter run `
  --dart-define=API_BASE_URL=http://10.0.2.2:8080/api/v1
flutter build apk --debug `
  --dart-define=API_BASE_URL=http://10.0.2.2:8080/api/v1
```

La ejecución comprobada fue flutter build apk --debug --dart-define=API_BASE_URL=http://10.0.2.2:18080/api/v1. Se generó build/app/outputs/flutter-apk/app-debug.apk, de 176.512.607 bytes. apksigner verify terminó con código 0 y aapt2 pudo leer el paquete. No se instaló el APK ni se verificó conectividad desde Android. Para el backend local final, que escucha 8080, recompílelo con 8080 antes de utilizarlo; no trate el APK de 18080 como una prueba de conexión.



Índice de C:\Users\compu\AppData\Local\Temp\vidia-anexo-b-

[directorio principal]

Nombre	Tamaño	Fecha de modificación
app-debug.apk	168 MB	12/10/26, 13:30:14
app-debug.apk.sha1	40 B	12/10/26, 13:30:16

Artefacto APK debug realmente generado en la copia temporal
Fuente: elaboración propia a partir de la consulta real. 12/10/2026 14:15 (Bolivia).

B.8.3. Firma release

android/app/build.gradle.kts exige android/key.properties para tareas Release. Los parámetros efectivos son storeFile, storePassword, keyAlias y keyPassword; pueden obtenerse mediante VIDIA_RELEASE_STORE_FILE, VIDIA_RELEASE_STORE_PASSWORD, VIDIA_RELEASE_KEY_ALIAS y VIDIA_RELEASE_KEY_PASSWORD. La presencia de variables no elimina la comprobación de existencia de key.properties y del keystore.

Se verificó la existencia de key.properties y del keystore, sin imprimir contenidos. Los valores efectivos de contraseña de firma estaban incompletos y no había variables VIDIA_RELEASE_* disponibles. Por ello no se ejecutó una compilación release ni se generó o reemplazó una firma de publicación.

```
flutter build apk --release `
  --dart-define=API_BASE_URL=https://trabajo-final-
diplomado.onrender.com/api/v1
```

El comando anterior queda pendiente y requiere credenciales autorizadas. Release exige HTTPS por ApiConfig. Después de una ejecución correcta, verifique build/app/outputs/flutter-apk/app-release.apk con apksigner y compare su certificado con el canal de actualización existente. Un APK debug no sustituye la publicación firmada.

[INSERTAR CAPTURA: compilación Android release y verificación de firma.]

Debe mostrar una compilación real y apksigner verify de app-release.apk, sin contenidos de key.properties, claves, contraseñas ni sustitución de la firma actual.

B.9. Despliegue y servicios públicos

B.9.1. Web en Render

Dato	Configuración versionada
Servicio / raíz	trabajo-final-diplomado-web; rootDir: frontend
Tipo	runtime: static
Construcción	npm ci && npm run build
Publicación	dist/frontend/browser
Navegación	Rewrite /* → /index.html
API compilada	environment.ts: backend HTTPS público bajo /api/v1

render.yaml declara solamente la aplicación web. Referencias antiguas a npm install no sustituyen su buildCommand vigente. No se accedió a la consola privada de Render ni se confirmó que sus campos actuales coincidan exactamente con el descriptor. No se incluyan secretos en un servicio estático.

B.9.2. Backend Docker y PostgreSQL

backend/Dockerfile construye con maven:3.9.9-eclipse-temurin-21-alpine, ejecuta dependency:go-offline y clean package -DskipTests, y copia el JAR a eclipse-temurin:21-jre-alpine. EXPOSE 8080 es metadato; el proceso usa server.port según SERVER_PORT, PORT o 8080.

```
docker build -t vidia-backend:revision backend
# Solo tras configurar una conexión accesible desde el contenedor:
docker run --rm --env-file .env `
  -p 8080:8080 vidia-backend:revision
```

Estos comandos Docker no se ejecutaron porque el daemon estaba indisponible. Para reproducir en Render, un responsable debe crear o revisar el servicio Docker con contexto backend y su Dockerfile, definir las variables de B.13 y el perfil prod, y comprobar la conexión JDBC con la base del proveedor. El backend no está declarado como servicio en render.yaml.

application-prod.yml declara app.storage.provider=supabase para el perfil prod. Confirme el valor efectivo: variables y argumentos pueden tener prioridad sobre archivos de configuración. SupabaseStorageConfiguration valida credenciales y endpoint HTTPS cuando se selecciona ese proveedor; si falta configuración, falla y no implementa una degradación automática. No seleccione disco local para ocultar la falta de almacenamiento persistente. No se comprobaron la versión, parámetros TLS, respaldos ni credenciales efectivos de PostgreSQL alojado.

B.9.3. Correo y almacenamiento

MAIL_PROVIDER=brevo-api selecciona BrevoApiCorreoProveedor: HTTPS a https://api.brevo.com/v3/smtp/email, encabezado api-key y cuerpo JSON con remitente, destinatario, asunto y texto. El segmento smtp de la ruta no convierte ese transporte en SMTP. No se enviaron correos para comprobar producción; se utilizaron mocks en tests y un servidor SMTP local no contactado para los arranques aislados.

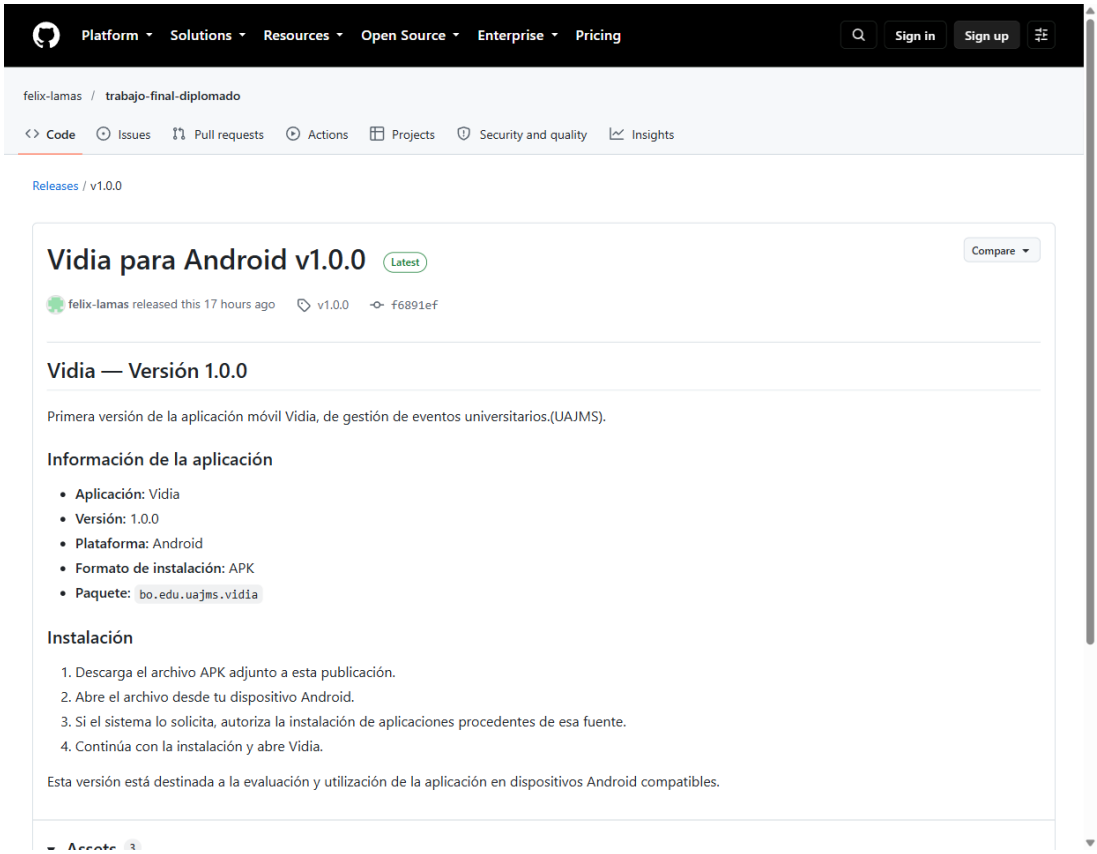
Supabase se consume mediante AWS SDK S3 2.55.10, endpoint HTTPS, access key / secret key y firma de solicitudes S3. El backend implementa guardar, obtener, comprobar existencia y eliminar objetos. Comprobantes y QR de pago usan referencias de objetos en PostgreSQL; certificados PDF se regeneran al descargarlos. No se probaron operaciones externas ni permisos del bucket productivo.

B.9.4. Distribución Android

La release pública v1.0.0 respondió correctamente y contiene un activo APK con enlace de descarga. El enlace del instalable se obtiene de la publicación existente; no se creó una release ni se subió el APK debug generado.

`https://github.com/felix-lamas/trabajo-final-diplomado/releases/tag/v1.0.0`

Antes de distribuir otra versión, incremente version en pubspec.yaml y compruebe compatibilidad de firma, API_BASE_URL HTTPS y pruebas. Mantenga versionCode creciente para actualizaciones Android. Estos son procedimientos de liberación, no acciones realizadas aquí.



Publicación Android existente en GitHub Releases
Fuente: elaboración propia a partir de la consulta real. 12/10/2026 13:34 (Bolivia).

[INSERTAR CAPTURA: configuración real de Render.]

Mostrar raíz, buildCommand, directorio publicado, Dockerfile, perfil y nombres de variables desde el panel autorizado, ocultando todos los valores sensibles. No se obtuvo acceso al panel.

B.10. Verificación posterior a la instalación o al despliegue

Comprobaciones públicas del 12/10/2026: consultas de lectura sin autenticación, sin carga ni modificaciones. El backend raíz responde 401 porque no constituye una página pública de bienvenida; salud, Swagger y OpenAPI tienen rutas públicas específicas.

Consulta	Resultado observado	Alcance
GET web pública	HTTP 200	Pantalla de acceso disponible.
GET backend /	HTTP 401	Protección de la ruta raíz.
GET /api/v1/salud	HTTP 200	Comprobación HTTP de salud; no prueba SQL del servicio productivo.
HEAD /api/v1/salud	HTTP 200 · cuerpo vacío	Compatible con monitorización HEAD; sin autenticación.
GET /swagger-ui/index.html	HTTP 200	Interfaz pública accesible.
GET /v3/api-docs	HTTP 200	Especificación publicada, 77 rutas.
GET /api/v1/usuarios/perfil	HTTP 401	Rechazo 401 sin token; no prueba todos los permisos.
GET GitHub Releases	HTTP 200	Publicación y activo APK existentes.

También se comprobaron localmente salud GET/HEAD, OpenAPI, Swagger y catálogo con 200 y perfil sin token con 401. Se utilizaron herramientas HTTP con validación TLS predeterminada para las URLs HTTPS públicas; no se inspeccionó la configuración interna de terminación TLS, JDBC ni la consola de Render.

En Swagger compruebe el servidor seleccionado: la interfaz distingue desarrollo local y servicio público. Para consultas publicadas seleccione la URL HTTPS de Render. No ejecute operaciones que cambien datos solo para verificar la disponibilidad.

```
curl.exe -I https://trabajo-final-diplomado.onrender.com/api/v1/salud
curl.exe https://trabajo-final-diplomado.onrender.com/api/v1/salud
curl.exe -i https://trabajo-final-
diplomado.onrender.com/api/v1/usuarios/perfil
```

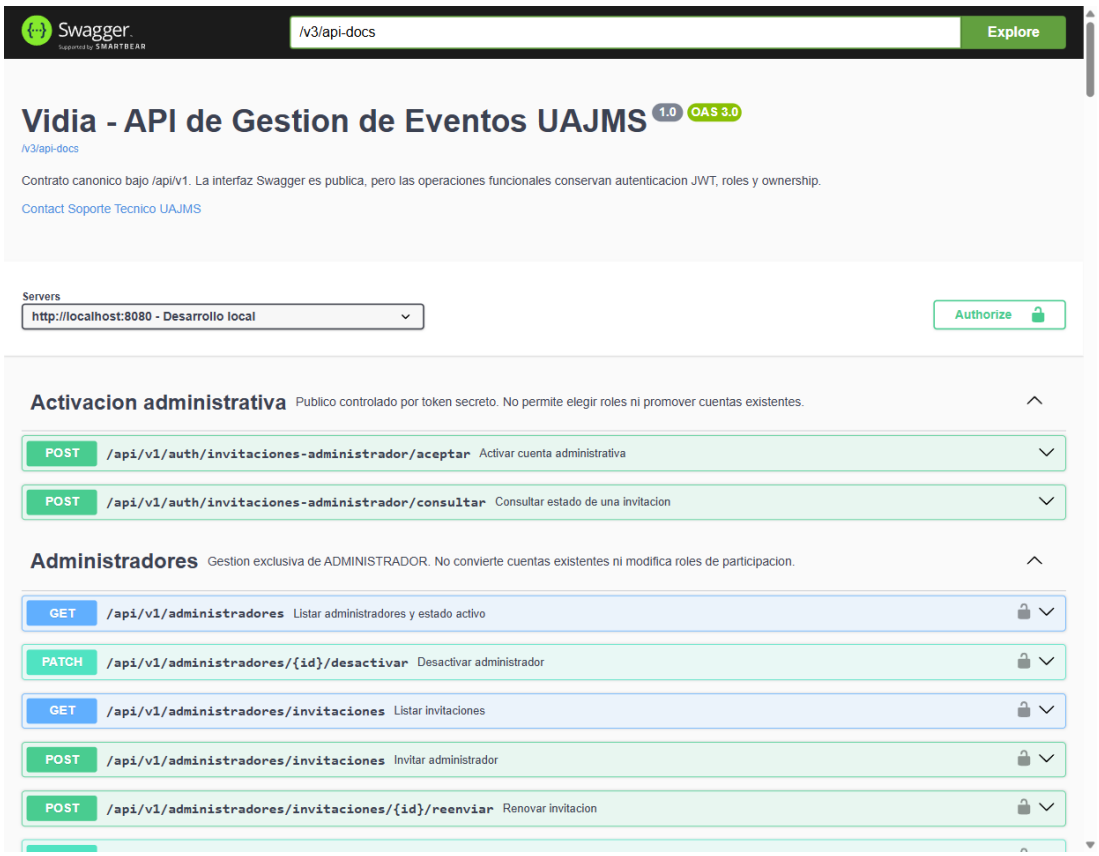
Estos ejemplos usan curl.exe para evitar el alias de PowerShell. Las comprobaciones registradas se hicieron con requests y navegador, no con esos comandos curl. Una

respuesta puntual no demuestra disponibilidad continua ni todos los módulos. No se verificó el SHA desplegado por Render.

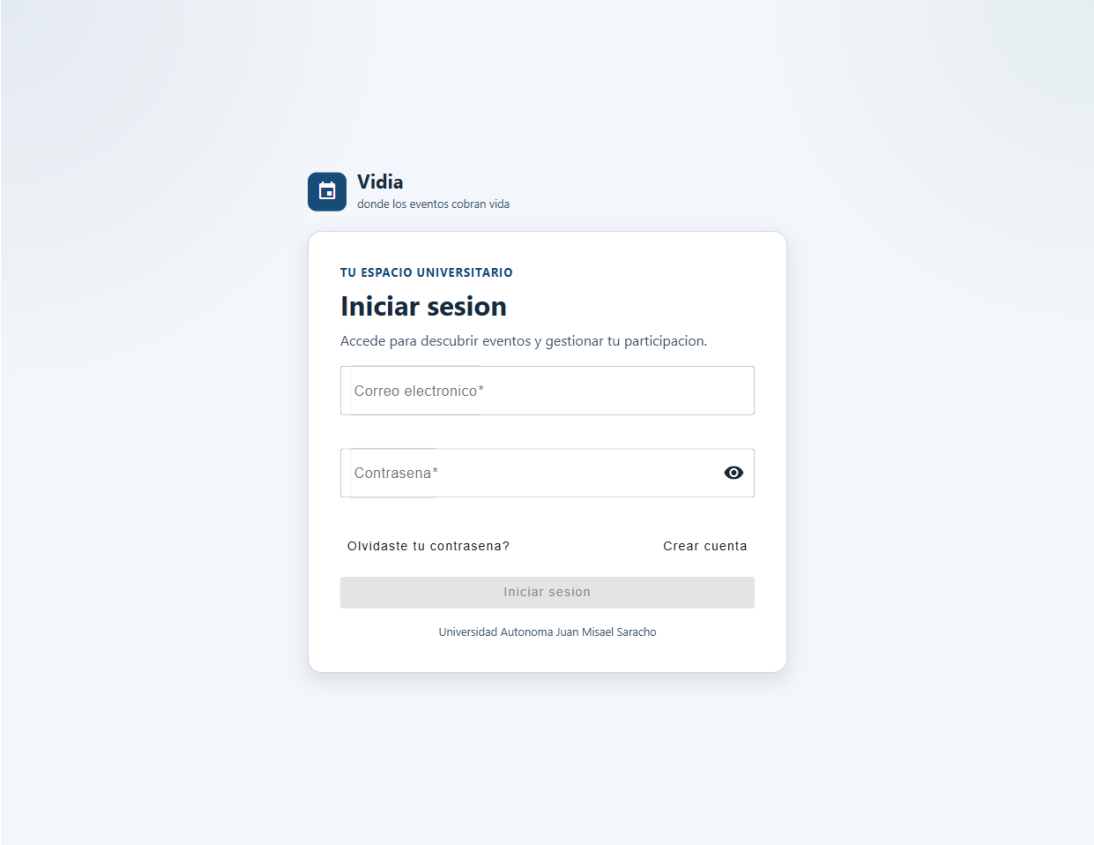
```
Dar formato al texto
```

```
{"estado":"UP"}
```

Respuesta pública real de salud del backend
Fuente: elaboración propia a partir de la consulta real. 12/10/2026 14:15 (Bolivia).



Swagger UI del backend publicado
Fuente: elaboración propia a partir de la consulta real. 12/10/2026 13:34 (Bolivia).



Página pública de acceso de Vidia
Fuente: elaboración propia a partir de la consulta real. 12/10/2026 13:34 (Bolivia).

B.11. Mantenimiento y seguridad

B.11.1. Control de cambios y liberaciones

1. Registre commit y árbol de trabajo de la versión candidata. Separe los cambios de aplicación de los secretos y artefactos de publicación.
2. Actualice dependencias en una copia o rama autorizada. Mantenga coherentes Angular/CLI, Node engines, Dart/Flutter y los archivos de bloqueo; no cambie todo a latest sin evaluar compatibilidad.
3. Ejecute backend verify, Angular test/build y Flutter test/analyze. Trate fallos, omisiones y advertencias por separado.

4. Pruebe migraciones nuevas sobre una copia y prepare respaldo antes de actualizar. No modifique V1–V5 ya aplicadas ni use repair/clean para ocultar discrepancias.
5. Construya el JAR o imagen y el APK release con firma autorizada. Registre hashes y configuración pública de API.
6. Después del despliegue autorizado, consulte salud, Swagger, catálogo y una matriz funcional segura. Registre la versión; no confunda la etiqueta Git con el código desplegado.

Estos son procedimientos recomendados para una liberación futura; no se actualizaron dependencias ni se ejecutó despliegue o rotación de secretos.

B.11.2. Respaldo y restauración

Se ejecutaron pg_dump -Fc y pg_restore sobre bases temporales del clúster 127.0.0.1:55439. Antes de habilitar demo, la base fuente contenía el esquema y una categoría ficticia; la base restaurada conservó esa fila y las cinco entradas de Flyway. No se certifican respaldos automáticos, recuperación de producción ni almacenamiento de archivos externos.

```
pg_dump -h 127.0.0.1 -p 5432 -U vidia_local `
  -d uajms_eventos -Fc -f respaldo_local.dump
# Cree antes una base VACÍA de restauración con administrador autorizado.
pg_restore -h 127.0.0.1 -p 5432 -U vidia_local `
  -d uajms_eventos_restauracion --no-owner --no-acl `
  --exit-on-error respaldo_local.dump
```

Los comandos anteriores son el procedimiento de referencia local; la ejecución registrada utilizó 55439. No incluya claves en la línea de comandos: use prompts o un mecanismo de credenciales restringido. La base de destino debe existir, estar aislada y ser propiedad del usuario autorizado. No restaure sobre la base publicada para probar el procedimiento.

Verifique tablas, restricciones, conteos e historial de Flyway. Pruebe también la aplicación restaurada antes de aprobar el plan institucional. Un dump de PostgreSQL no incluye automáticamente los objetos de Supabase ni los archivos legacy locales.

Se necesita inventario, retención, respaldo separado de objetos y prueba de restauración de esas referencias.

[INSERTAR CAPTURA: respaldo y restauración locales.]

Capturar pg_dump, pg_restore y consultas de comparación del clúster aislado. La ejecución y su alcance están en migraciones_y_restauracion.json; no insertar datos o credenciales de producción.

B.11.3. Operación segura

Proteja DB_PASS, JWT_SECRET, BREVO_API_KEY, credenciales S3 y firma Android fuera de Git. Defina responsables y un procedimiento de rotación; la rotación JWT puede invalidar tokens existentes y debe coordinarse con los clientes. No se efectuó una rotación en esta revisión.

Revise errores de base, migraciones, correo y archivos sin registrar contraseñas, tokens o cuerpos sensibles. Correlacione incidentes por fecha y operación. Los secretos efectivos, registros privados y retención de Render no se inspeccionaron.

Salud permite GET y HEAD públicos. Puede utilizarse para vigilancia periódica, pero hacen falta una ventana de observación, registros y responsables para afirmar disponibilidad. Los límites de autenticación del código actual son por instancia y en memoria; su configuración productiva no se verificó.

Mantenga los buckets privados y permita operaciones de archivos mediante el backend. No elimine objetos por no aparecer en una pantalla: contraste referencias, estados y retención. En el proveedor local, uploads y archivos legacy necesitan persistencia o respaldo propios; el disco efímero no sirve como respaldo institucional.

B.12. Diagnóstico de problemas frecuentes

Problema	Comprobación / causa posible	Acción segura
Java/Maven/psql no reconocidos	Comprobar --version y PATH. psql no estaba en PATH, aunque los binarios existían.	Use la ruta instalada o ajuste PATH de la sesión; no sustituya versiones sin revisar manifiestos.
Dependencias no se descargan	Revisar red, repositorio, caché y compatibilidad de Node/Dart. No se observó fallo en ci/pub get/verify.	Conservar lockfiles; repetir cuando se resuelva la causa. No borrarlos para ocultar incompatibilidades.
PostgreSQL no conecta	Verificar host, puerto, DB_URL, usuario y autenticación. No probar credenciales publicadas.	Utilizar una base local y comprobar SELECT 1. Revisar TLS del proveedor si se exige.
Puerto 8080 ocupado	Observado al iniciar el JAR mientras Maven seguía activo.	Identifique el dueño del proceso; detenga solo el servicio propio o cambie SERVER_PORT y la API del cliente.
Variables requeridas ausentes	DB_USER, DB_PASS, JWT_SECRET; la selección Supabase requiere credenciales externas.	Completar valores por el canal protegido. Nunca copiar secretos al frontend o al manual.

Problema	**Comprobación / causa posible**	**Acción segura**
Frontend no comunica con API	Comparar environment.dev.ts/environment.ts, build configuration y CORS.	Confirmar origen exacto y puerto. Una variable del hosting no reemplaza una constante compilada.
Android no comunica con API	Revisar API_BASE_URL, /api/v1, emulador frente a teléfono físico, HTTP debug/HTTPS release.	Recompilar con dirección accesible; no cambiar manifest de release para permitir HTTP.
Flyway falla	Consultar versión, checksum y error. Baseline 0 no acredita esquema histórico compatible.	Probar una copia y migración forward-only; no editar SQL aplicado ni limpiar datos.
Analyze / builds con avisos	17 infos en Flutter; presupuestos de tamaño en Angular.	Registrar impacto y planificar corrección. No declarar una ejecución sin observaciones.
APK release bloqueado	Valores efectivos de firma incompletos observados.	Obtener credenciales autorizadas; no generar otra firma ni etiquetar debug como release.
Docker no responde	Cliente presente; daemon sin conexión observado.	Restablecer el entorno autorizado. El código pudo probarse con PostgreSQL nativo, sin fingir un contenedor ejecutado.

Problema	Comprobación / causa posible	Acción segura
Correo/Storage falla	Claves, remitente, endpoint HTTPS o permisos pueden faltar; no se probaron integraciones externas.	Verificar de forma controlada con cuenta/bucket de prueba y permiso. No cambiar a almacenamiento efímero para esconder el fallo.
Enlaces apuntan a localhost	Comparar URLs frontend y CERTIFICATE_VERIFICATION_BASE_URL. Se observó este problema en la consulta del certificado del Anexo A.	Revisar configuración autorizada y nuevas emisiones en un entorno de prueba; no alterar certificados existentes aquí.
Servicio publicado no disponible	Consulta puntual, red, suspensión o fallo del hosting: causas posibles, no demostradas.	Registrar fecha y resultado, consultar estado/logs autorizados y reintentar; no atribuir una causa sin evidencia.

B.13. Variables de entorno y configuración segura

Las siguientes variables se localizaron en application.yml, application-prod.yml, anotaciones @Value, Compose y Gradle. No todas se requieren en todos los perfiles. .env.example omite algunas presentes en application.yml; revise el contrato actual antes de preparar un ambiente. No se copiaron valores privados del hosting.

Servidor

Variable(s)	Uso y condición
APP_NAME SERVER_PORT / PORT	Opcionales; nombre de aplicación y puerto (8080 como último fallback).

Perfiles

Variable(s)	Uso y condición
SPRING_PROFILES_ACTIVE DEMO_PASSWORD	dev por defecto; demo activa seed y exige clave propia. prod declara Supabase; verificar el valor efectivo. DEMO_PASSWORD no es clave universal de producción.

Base

Variable(s)	Uso y condición
DB_URL DB_HOST / DB_PORT / DB_NAME DB_USER / DB_PASS	URL alternativa o componentes (localhost, 5432, uajms_eventos). Usuario/clave obligatorios sin default; solo backend.

JPA

Variable(s)	Uso y condición
JPA_DDL_AUTO JPA_SHOW_SQL JPA_FORMAT_SQL	update por defecto; validate usado en ensayos. Controles opcionales de diagnóstico; evitar datos sensibles en SQL/logs.

JWT

Variable(s)	Uso y condición
JWT_SECRET JWT_EXPIRATION	Secreto obligatorio y seguro, Base64; expiración en milisegundos (86400000 como default del código). Valores efectivos de producción no verificados.

CORS

Variable(s)	Uso y condición
CORS_ALLOWED_ORIGINS	Orígenes separados por coma; reflejar dominios reales. Local probado: http://localhost:4200. No es una contraseña ni configura la URL Angular.

Intentos

Variable(s)	Uso y condición
AUTH_RATE_IDENTIFIER_LIMIT AUTH_RATE_OPERATION_LIMIT AUTH_RATE_WINDOW_SECONDS AUTH_RATE_MAX_IDENTIFIERS	Defaults 10, 120, 60 segundos y 10000 identificadores. Estado en memoria/por instancia. Producción no confirmada.

Invitación admin

Variable(s)	Uso y condición
ADMIN_INVITATION_EXPIRATION_HOURS ADMIN_INVITATION_RESEND_MINUTES	Opcionales; 24 horas y 5 minutos por defecto. Solo se documenta configuración; no se enviaron invitaciones publicadas.

Enlaces

Variable(s)	Uso y condición
FRONTEND_RESET_PASSWORD_URL FRONTEND_VERIFY_EMAIL_URL FRONTEND_ADMIN_ACTIVATION_URL CERTIFICATE_VERIFICATION_BASE_URL	Configurar URLs del frontend correcto. Defaults locales pueden ser inapropiados en Render. La base pública de certificados es /verificar-certificado, no la URL API.

Correo API

Variable(s)	Uso y condición
MAIL_PROVIDER MAIL_FROM / MAIL_FROM_NAME BREVO_API_KEY	brevo-api por defecto. El envío real requiere API key y remitente autorizado. La clave se lee en BrevoApiCorreoProveedor, no directamente en la sección YAML de Storage.

SMTP alternativo

Variable(s)	Uso y condición
SPRING_MAIL_HOST / SPRING_MAIL_PORT SPRING_MAIL_USERNAME / SPRING_MAIL_PASSWORD SPRING_MAIL_PROPERTIES_ MAIL_SMTP_AUTH SPRING_MAIL_PROPERTIES_ MAIL_SMTP_STARTTLS_ENA BLE	Solo cuando se seleccione smtp. No describir Brevo API como transporte SMTP. SMTP no se contactó en el arranque local.

Tiempo SMTP

Variable(s)	Uso y condición
SPRING_MAIL_PROPERTIES_ MAIL_SMTP_CONNECTIONTI MEOUT SPRING_MAIL_PROPERTIES_ MAIL_SMTP_TIMEOUT SPRING_MAIL_PROPERTIES_ MAIL_SMTP_WRITETIMEOUT	Opcionales; límites de espera de la alternativa SMTP. No son tiempos de rendimiento de la API Vidia.

Archivos

Variable(s)	Uso y condición
APP_STORAGE_PROVIDER UPLOADS_BASE_DIR	local por defecto general; uploads como raíz relativa local. el perfil prod declara supabase; comprobar el proveedor efectivo. Directorio local debe respaldarse si conserva referencias legacy.

S3 privado

Variable(s)	Uso y condición
SUPABASE_STORAGE_ENDPOINT SUPABASE_STORAGE_REGION SUPABASE_STORAGE_BUCKET SUPABASE_STORAGE_ACCESS_KEY SUPABASE_STORAGE_SECRET_KEY	Requeridos por proveedor supabase: endpoint HTTPS, región/bucket y credenciales S3. No utilizar anon key del cliente ni credenciales en Angular/Flutter.

Tiempo S3

Variable(s)	Uso y condición
SUPABASE_STORAGE_CONNECTION_TIMEOUT_MS SUPABASE_STORAGE_API_TIMEOUT_MS	Opcionales; defaults 5000 y 20000 ms. No prueban disponibilidad del servicio.

Cargas

Variable(s)	Uso y condición
MAX_FILE_SIZE MAX_REQUEST_SIZE	Límites multipart del servidor, 10MB por defecto. Los comprobantes tienen además una regla funcional de 5 MB; no confundir ambos niveles.

pgAdmin

Variable(s)	Uso y condición
PGADMIN_EMAIL PGADMIN_PASSWORD	Solo el servicio opcional pgAdmin de Compose; no necesarios para ejecutar Spring con PostgreSQL nativo.

Android

Variable(s)	Uso y condición
API_BASE_URL VIDIA_RELEASE_STORE_FILE VIDIA_RELEASE_STORE_PASSWORD VIDIA_RELEASE_KEY_ALIAS VIDIA_RELEASE_KEY_PASSWORD	API_BASE_URL es dart-define público, no secreto. Los otros parámetros son de firma Gradle; key.properties y el keystore deben existir. Nunca incluirlos en assets ni Git.

El .env importado por Spring se trata como archivo de propiedades, no como un script de shell: evite comillas agregadas al valor y expansión de comandos. No presume carga automática de .env por Angular o Flutter. Mantenga variables sensibles fuera del control de versiones y limite el acceso a archivos locales.

En Render deberán configurarse, por el responsable autorizado, DB_USER/DB_PASS/DB_URL, JWT_SECRET, almacenamiento externo, correo y URLs del frontend. No se conocen sus valores efectivos; este manual no los rellena por suposición.

La selección del proveedor se basa en propiedades (@ConditionalOnProperty). No se localizó una prohibición adicional por perfil en FilesystemStorageService; por ello la configuración efectiva y la política de persistencia deben verificarse por el responsable, sin suponer que un comentario del código garantiza ese control.

B.14. Limitaciones y evidencias pendientes

Comprobación	**Estado y evidencia**
Backend	verify 0; 590 aprobadas, una omitida; arranque Maven y JAR, Flyway y acceso demo locales comprobados.
Angular	npm ci, 316 pruebas, build y servidor local correctos; advertencias de presupuestos conservadas.
Flutter	pub get y 139 pruebas correctos; analyze 1 con 17 infos. Doctor advierte licencias. APK debug y firma del artefacto verificados; no instalado.
Docker	Descriptor Compose validado; daemon no disponible. Construcción y ejecución de imagen pendientes.
Release Android	APK publicado consultado; generación release nueva pendiente por firma incompleta. No se comprobó identidad binaria de una APK instalada.
Producción	Web, salud, Swagger, OpenAPI y release accesibles; perfil anónimo 401. Sin acceso a panel, base, secretos, SHA ni métricas continuas.

Comprobación	Estado y evidencia
Restauración	Esquema y una categoría ficticia restaurados localmente antes de activar demo; respaldo de archivos/S3 y restauración institucional pendientes.

[INSERTAR CAPTURA: construcción y arranque del contenedor backend.]

Debe mostrar docker build y ejecución real con una base aislada. Hoy el daemon no estaba disponible; el arranque del JAR no sustituye esa prueba.

[INSERTAR CAPTURA: correo y almacenamiento en un entorno de prueba.]

Mostrar resultados reales con cuentas y bucket aislados autorizados, sin secretos ni envíos a usuarios reales. La prueba Supabase opt-in fue omitida deliberadamente.

Las capturas públicas corresponden a disponibilidad puntual. Los registros de CLI son evidencia ejecutada, pero no se obtuvieron capturas de una terminal gráfica para todos ellos. Los marcadores solicitan imágenes reales, no pantallas reconstruidas a partir de texto.

No se declaran reproducidos de extremo a extremo Docker, la consola Render, servicios externos, un APK release nuevo, la conectividad del APK debug ni una recuperación institucional. Para cerrar esos puntos se necesitan permisos, credenciales autorizadas, ambiente de ensayo y responsables operativos.

Fuentes principales: Anexo B de Monografia_Vidia_Final.docx; lineamientos técnicos y formato institucional de docs/documentoFinal/; backend/pom.xml y application*.yml; migraciones V1–V5; Dockerfile; docker/docker-compose.yml; render.yaml; package.json, package-lock.json y angular.json; pubspec.yaml/lock; app_config.dart y archivos Gradle. README conserva referencias históricas que se sustituyen en este manual por la configuración comprobada, sin modificar ese archivo.